

医療AIプラットフォーム CI/CD環境構築ガイド

 GitHub Actions +  Amazon ECR +  Amazon EKS (Fargate)

 2026年01月15日

 ap-northeast-1 (東京リージョン)

 Dev / Staging / Production

 AWS Well-Architected 準拠

目次

CI/CD環境構築ガイド アジェンダ

パイプライン戦略

01 CI/CDパイプライン概要

02 Git戦略（GitFlow）

03 CI（継続的インテグレーション）

04 CD（継続的デリバリー）

デプロイメント

05 環境昇格フロー

06 ECRイメージ管理

07 EKSデプロイメント戦略

品質とガバナンス

09 環境別設定管理

10 品質ゲート

11 ロールバック戦略

12 セキュリティ

運用と開発者支援

13 Infrastructure as Code

14 監視・通知

15 開発環境セットアップ

16 メトリクス・KPI

17 まとめ



Commit & PR

開発者がコードを
Commit
PR作成でトリガー



GitHub

ソース管理
コードレビュー



Actions

Lint / Test / Build
Security Scan



Amazon ECR

イメージPush
脆弱性スキャン



Amazon EKS

Fargateへデプロイ
(Dev/Stg/Prod)



Monitor

CloudWatch
Slack通知

✓ パイプライン設計のキーポイント

1 徹底した自動化

ビルド、単体テスト、セキュリティスキャン、デプロイメントまでの一連のプロセスをGitHub Actionsで完全自動化。手作業によるミスを排除し、一貫性を担保します。

2 Security First

Trivy/Snykによる脆弱性スキャンをパイプラインに組み込み。署名付きイメージのみをデプロイ許可し、AWS Secrets Managerで機密情報を厳格に管理します。

3 完全なトレーサビリティ

Git SHAとタイムスタンプを含む不変のタグ戦略（Immutable Tags）を採用。どのコードがどの環境で稼働しているかを常に追跡可能にします。



🔗 ブランチ戦略

main **Protected**
本番環境用。直接コミット禁止。常にリリース可能な状態を維持。

hotfix/*
本番の緊急修正用。mainから分岐し、mainとdevelopにマージ。

release/*
リリース準備用。バージョンの凍結と最終テスト。

develop
開発の統合ブランチ。Staging環境への自動デプロイ元。

feature/*
機能開発用。developから分岐し、PR経由でマージ。短寿命推奨。

📋 プルリクエスト（PR）ルール

👥 レビュー体制

必須 コードレビュー **2名以上** の承認 (Approve)

推奨 WIP (Work In Progress) の場合は Draft PR を作成

✅ 自動チェック (Status Checks)

必須 全ての CI ジョブ (Lint, Unit Test, Security Scan) の通過

必須 コンフリクトの解消とベースブランチの最新化

🔗 マージ戦略

必須 **Squash and merge** を使用し、コミット履歴を簡潔に保つ

必須 マージ後のブランチは自動削除



Push / PR



Lint & Test



Security Scan



Build Image



Push ECR

.github/workflows/ci.yml

YAML

```
name: CI Pipeline

on:
  push:
    branches: [ "develop", "release/*" ]
  pull_request:
    branches: [ "main", "develop" ]

jobs:
  quality_gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Lint Code
        run: |
          npm ci
          npm run lint # ESLint check
```

1 厳格な品質ゲート

Lintによる静的解析とユニットテスト（カバレッジ80%以上）を必須化。品質基準を満たさないコードはマージおよびビルドフェーズに進めません。



環境ごとに最適化されたデプロイ戦略を採用し、開発スピードと本番環境の安全性を両立させます。

</> Development

AUTO

開発効率最優先の高速サイクル

完全自動デプロイ
feature → develop マージで即時反映

機能検証
開発者自身による動作確認環境

Rolling Update
ダウンタイムを許容し迅速更新

AWS Fargate

Spot Instances

📁 Staging

AUTO / GATE

本番同等の品質保証環境

リリースブランチ連動
develop → release 作成でデプロイ

QA & 性能テスト
E2Eテスト、負荷試験の実施

データ整合性確認
DBマイグレーション検証

Prod-Like Config

Integration Test

🚀 Production

MANUAL APPROVAL

高可用性と安全性の担保

承認プロセス
CAB/マネージャー承認後に実行

Blue/Green Deployment
新旧環境並行稼働・即時切替

Canary Release
10%→50%→100% 段階的公開対応

Multi-AZ

Auto Scaling



Development

feature → develop

特徴

開発者による迅速な機能検証環境。コミット毎に自動デプロイされ、フィードバックサイクルを高速化。

GUARDRAILS (自動)

🔧 Unit Test Coverage > 80%

🔍 Static Analysis (Lint) Pass

🚢 Docker Build Success



Staging

develop → release/*

特徴

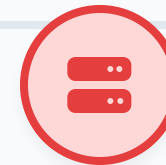
本番相当の構成でのQA及び性能テスト。リリース候補版（RC）としての品質評価を実施。

GUARDRAILS (品質ゲート)

🛡️ Container Vuln Scan (High=0)

🕒 Performance Test Pass

📋 QA Sign-off / Integration Test



Production

release/* → main

特徴

実際のユーザーが利用する環境。厳格な変更管理と、無停止デプロイ戦略（Blue/Green）を適用。

GUARDRAILS (承認・統制)

🔑 CAB Approval / Manual Gate

🔒 Signed Image Verification

🔄 ChangeLog / Ticket Link



📁 リポジトリ構成とタグ戦略

📁 リポジトリ構成

api-service

バックエンドAPIサービス



ml-inference-service

AI推論エンジン



worker-service

非同期処理ワーカー



🔖 タグ命名規則 (Immutable Tags)

{env} - {git-sha} - {timestamp}

prod-a3f4b2c-20260115-120000

📌 環境、Gitコミットハッシュ、日時を含めることで、完全なトレーサビリティと一意性を確保

🛡️ セキュリティとライフサイクル

🔍 イメージスキャン (脆弱性対策)

- 1 **プッシュ時自動スキャン**
ECRへのプッシュトリガーでTrivy/Clairを実行
- 2 **デプロイブロック基準**
Critical/Highレベルの脆弱性が検出された場合、ビルドを失敗させデプロイを阻止

♻️ ライフサイクルポリシー

保持ルール

Cost Optimization

- ✓ タグ付きイメージ: **最新 20 世代** を保持
- ✓ タグなし(untagged): **1日経過後** に自動削除

SIGNED IMAGES

🔗 Cosign / Sigstore によるイメージ署名と検証を推奨



サービスの要件、リスク許容度、インフラコストに応じて最適なデプロイ戦略を選択します。



Rolling Update

DEFAULT STRATEGY

概要

Podを1つずつ（またはバッチで）新しいバージョンに入れ替え。ダウンタイムゼロだが、新旧バージョンが一時的に混在する。

特徴

- ✓ 追加リソース消費が最小限
- ✓ Kubernetes標準機能で実装容易
- ✓ ロールバックに時間がかかる可能性

✓ 推奨: Development / Staging



Blue/Green

SAFE & INSTANT SWITCH

概要

新環境（Green）を構築し、テスト完了後にトラフィックを一括切り替え。旧環境（Blue）は待機させる。

特徴

- ✓ 切り替え時のダウンタイムほぼゼロ
- ✓ 問題発生時の即時ロールバックが可能
- ✓ 一時的に2倍のリソースが必要

★ 推奨: Production (標準)



Canary Release

GRADUAL ROLLOUT

概要

少量のトラフィック（例: 10%）のみを新版に流し、メトリクスを確認しながら段階的に比率を上げる。

特徴

- ✓ 本番影響を最小限に抑えたテスト
- ✓ 自動分析による異常検知と中断
- ✓ 設定・運用が複雑（Argo Rollouts等）

⚠ 推奨: High Risk Changes



本番環境の安全性と可用性を最大化するための、高度なデプロイメント戦略と自動ロールバック機構。

Blue/Green

SAFE SWAP

即時切り替えとロールバック

- ✓ **Green検証**
本番同等トラフィックで最終確認後に切り替え
- 🕒 **待機期間**
切り替え後、Blue環境を一定期間保持
- 🔄 **即時復旧**
問題発生時はトラフィックをBlueに戻すだけ

Argo Rollouts

TargetGroupBinding

Canary Release

PHASED

段階的なトラフィック移行

- ▶ **Phase 1: 10% (15分)**
初期ユーザーへの公開とメトリクス監視
- ▶ **Phase 2: 50% (30分)**
安定稼働確認後、半数のトラフィックへ拡大
- 🚩 **Phase 3: 100%**
全トラフィック移行完了

Flagger

Istio / App Mesh

監視 & 自動復旧

AUTO HEAL

異常検知による自動防御

- 📈 **主要監視指標**
エラー率、レイテンシ(P95/P99)、Podヘルス
- 🔧 **自動判定**
Prometheusメトリクスに基づき進行/中止判断
- 🛡️ **自動ロールバック**
異常検知時、即座に旧バージョンへ切り戻し

Prometheus

CloudWatch



機密情報とアプリケーション設定を分離し、環境ごとに安全かつ柔軟に管理する戦略を定義します。

機密情報管理

SECURE

認証情報・APIキーの厳格な管理

 **AWS Secrets Manager**
DBパスワード等を一元管理・自動ローテーション

 **GitHub Secrets**
CI/CDパイプライン用の資格情報管理

 **External Secrets Operator**
K8s上のSecretリソースとして同期

Encryption At Rest

No Hardcoding

アプリ設定

FLEXIBLE

非機密パラメータの環境差分管理

 **ConfigMap活用**
環境依存の設定値をK8sオブジェクト化

 **環境変数注入**
コンテナ起動時に値を上書き設定

 **バージョン管理**
設定変更履歴をGitで追跡可能に

Immutable Config

Hot Reload

Helm運用

TEMPLATE

マニフェストのテンプレート化

 **values.yaml分離**
values-dev.yaml / values-prod.yaml

 **共通チャート利用**
DRY原則に基づきテンプレートを共通化

 **差分最小化**
環境間の設定差異を可視化・管理

Helm v3

Template Engine



Gate: Dev → Staging

Development Integration Check



CIパイプライン全通過

AUTOMATED

Lint、Unit Test (Coverage $\geq$ 80%)、ビルドがすべて成功していること。



コードレビュー承認

MANUAL

シニアエンジニアを含む**2名以上**のApproveが必須。コンフリクト解消済みであること。



コンテナ脆弱性スキャン

AUTOMATED

ECR Push時にTrivy/Snykを実行。**CRITICAL/HIGH**レベルの脆弱性がゼロであること。

✓ 自動化率: 70% (速度重視)

Gate: Staging → Production

Production Release Readiness



QA承認 (Sign-off)

MANUAL

統合テストおよびE2Eテスト完了報告。既知の重大バグが残っていないこと。



性能基準クリア

AUTOMATED

負荷試験にて目標RPSを達成し、P95レイテンシおよびエラー率が基準値以下であること。



CAB承認 (Change Advisory Board)

MANUAL

変更計画書、ロールバック手順、リスク評価の承認。リリース判定会議でのGoサイン。



セキュリティ最終監査

AUTOMATED

署名付きイメージ (Cosign) の検証。IaCセキュリティスキャン (tfsec) の通過。

🛡️ 安全性重視 (厳格な承認プロセス)



障害発生時のダウンタイムを最小限に抑えるため、自動検知と迅速な復旧プロセスを定義しています。

自動ロールバック

TRIGGER

異常検知時の即時自動復旧

ヘルスチェック失敗
Startup/Liveness Probe連続失敗時

エラー率超過
5xxエラー率 > **5%** (5分間平均)

レイテンシ劣化
P95応答時間 > **2000ms**

CloudWatch Alarm

Argo Rollouts

手動ロールバック

OPERATION

予期せぬ不具合への緊急対応

コマンド実行
`kubectl rollout undo deployment/api`

目標復旧時間 (RTO)
検知から**5分以内**に正常系へ復帰

GitOps同期一時停止
ArgoCD AutoSyncを一时无効化

Kubectl

ArgoCD CLI

安全性担保施策

POLICY

ロールバックを成功させる前提条件

イメージタグ固定
latestタグ禁止、不変タグ(Immutable)運用

DB前方互換性
スキーマ変更はコード変更と分離して実施

直前バージョン保持
ECR/Helmリポジトリにn-1世代を常時確保

Immutable Tag

Backward Compat



DevSecOpsアプローチを採用し、開発プロセスの初期段階から自動化されたセキュリティチェックを組み込みます。

</> SAST

STATIC ANALYSIS

コード品質と静的解析

SonarQube

品質ゲート: Aランク必須
バグ・脆弱性・Code Smellの検出

CodeQL

GitHub Advanced Security連携
主要言語の深層解析をPR毎に実行

Quality Gate

Auto Review

Container

VULNERABILITY

イメージと依存関係の保護



Trivy

OS/ライブラリの既知脆弱性スキャン
CRITICAL/HIGHレベルでビルド停止



Snyk

OSSライセンス違反と依存関係チェック
修正PRの自動作成機能

Image Scan

Dep Check

Secrets

CREDENTIALS

機密情報の厳格な管理



ハードコード禁止

git-secrets / trufflehogによる
コミット時のシークレット混入防止



Secrets Manager一元管理

環境変数は実行時に注入
KMSによる暗号化と自動ローテーション

Encryption

Rotation



Infrastructure

Terraform Provisioning (VPC, EKS, RDS)



Application

ArgoCD GitOps (Manifests, Helm)

terraform/environments/prod/main.tf

</> HCL

```
terraform {
  backend "s3" {
    bucket      = "medical-ai-tfstate-prod"
    key         = "platform/terraform.tfstate"
    region      = "ap-northeast-1"
    dynamodb_table = "terraform-locks"
    encrypt     = true
  }
}
```

```
module "eks_cluster" {
  source          = "../modules/eks"
  cluster_name    = "medical-ai-prod"
  cluster_version = "1.28"

  vpc_id      = module.vpc.vpc_id
  subnets    = module.vpc.private_subnets
```

1 Terraformによるインフラ管理

環境ごとのモジュール化設計とS3バックエンドによるState管理を徹底。
DynamoDBによるロック機構で競合を防ぎ、CIパイプラインでの自動plan/apply
を実現します。

2 GitOps (ArgoCD) の採用

KubernetesマニフェストをGitリポジトリで宣言的に管理。ArgoCDがGitの状態を
クラスタに自動同期し、ドリフト（構成乖離）を即座に検出・修正します。



パイプラインの健全性を可視化し、異常検知時に即座に通知を行うことで、開発サイクルの継続性を維持します。

監視項目

METRICS

パフォーマンス指標の常時測定

🔗 **パイプライン成功率**
ビルド/テスト/デプロイの成功比率

🕒 **実行時間計測**
ビルド時間、キュー待ち時間の推移

🚀 **デプロイ頻度・時間**
環境ごとのデプロイ所要時間と回数

CloudWatch

GitHub API

通知先

CHANNELS

アクションに応じた適切な通知

🔔 **Slack通知**
成功/失敗、承認待ち、ロールバック通知

✉️ **Email通知**
Productionデプロイ完了、重要アラート

📄 **Jira連携（Optional）**
ビルド失敗時のチケット自動起票

ChatOps

SNS

運用ダッシュボード

VISUALIZATION

全体状況の一元管理

🐙 **GitHub Actions**
ワークフロー実行履歴とログ詳細

🔗 **ArgoCD UI**
K8sリソース状態と同期ステータス

📊 **CloudWatch Dashboard**
システムメトリクスとログの統合表示

Unified View

Real-time



開発者が即座に価値提供を開始できるよう、標準化されたローカル環境と充実したオンボーディング資料を提供します。

Local Dev

ENVIRONMENT

コンテナベースの標準開発環境

 **Docker Compose**
DB、API、Workerを一括起動・連携

 **Minikube / Kind**
ローカルでのK8sマニフェスト動作検証

 **Hot Reloading**
コード変更の即時反映による高速開発

Docker

K8s Local

Onboarding

DOCS

スムーズな参画支援リソース

 **クイックスタートガイド**
「10分でHello World」セットアップ手順

 **開発標準・規約**
コーディング規約、PRテンプレート

 **アーキテクチャ図**
システム全体像とデータフローの解説

Wiki/Confluence

README.md

Support

HELP

トラブルシューティングとFAQ

 **FAQデータベース**
よくある環境構築エラーと解決策

 **デバッグツール**
ログ解析・トレース用のスクリプト群

 **サポートチャンネル**
SlackでのリアルタイムQ&Aサポート

Common Errors

Slack Bot



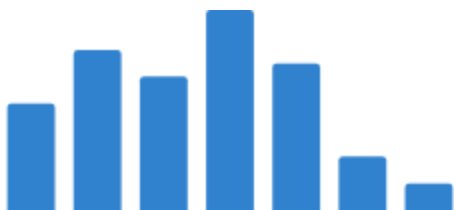
メトリクス・KPI (Four Keys)

Deployment Frequency & Stability Metrics

DEPLOYMENT FREQUENCY

10 /week

🎯 Target: 週10回以上



LEAD TIME FOR CHANGES

< 1 hour

🎯 Target: 1時間以内



CHANGE FAILURE RATE

< 5 %

🎯 Target: 5%以下



MEAN TIME TO RESTORE

< 30 min

🎯 Target: 30分以内



CONTINUOUS IMPROVEMENT CYCLE



Measure

月次レビューで現状の
Four Keysを測定・可視化



Analyze

ボトルネック特定と失敗
からの学習（ポストモー
テム）



Optimize

パイプラインの自動化強
化・並列化・不要工程の
削除



Standardize

改善策をテンプレート化
し全チームへ横展開